Step 1:

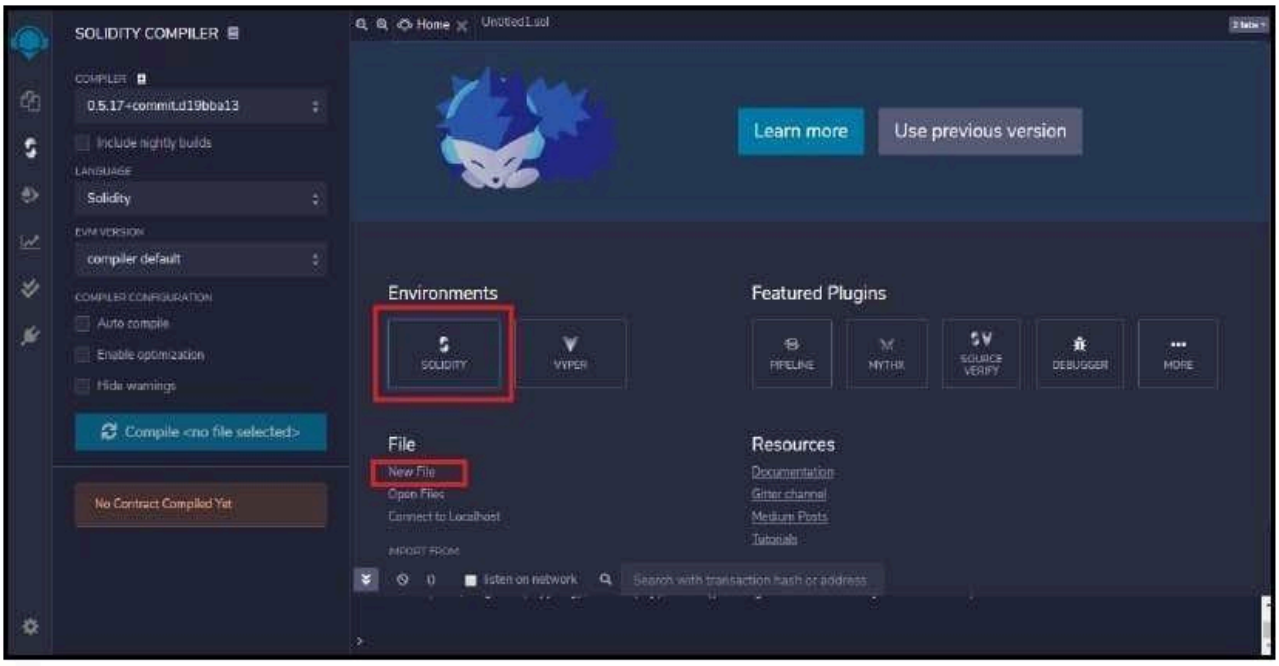
Open Remix IDE on any of your browsers, select on the

New File and click on*Solidity* to choose the environment.

Step 2:

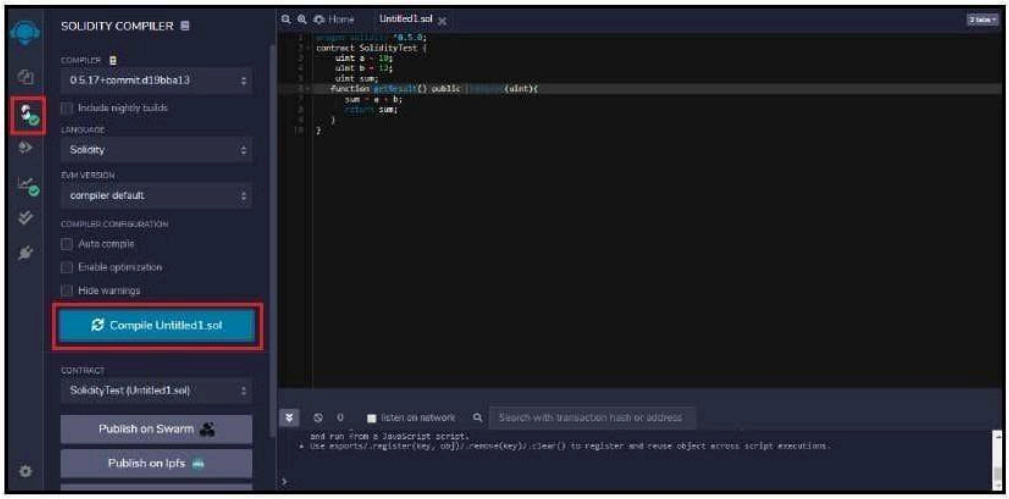
Write the Smart contract in the code section, and click the

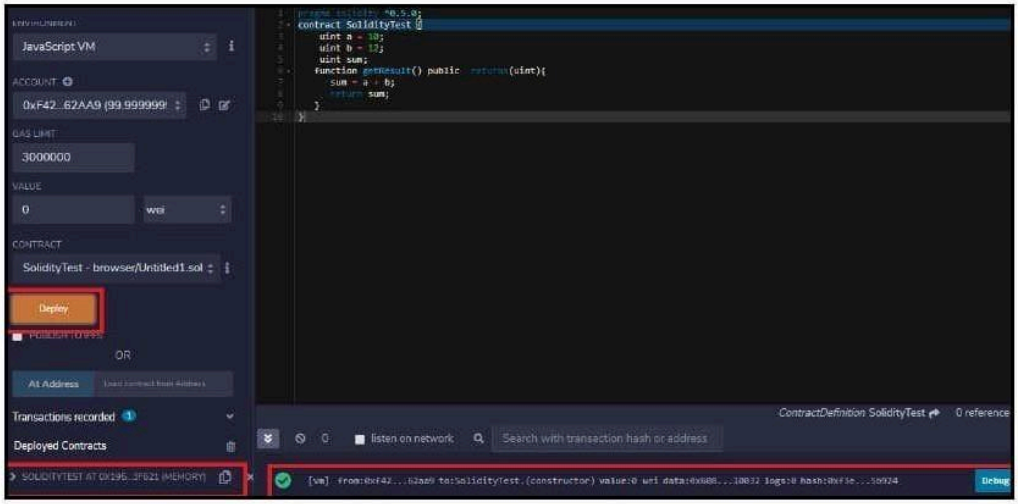
Compile button under the Compiler window to compile the contract.



Step 3:

To execute the code, click on the *Deploy button* under Deploy and Run Transactions window.





Step 4:

After deploying the code click on the method calls under the drop-down of

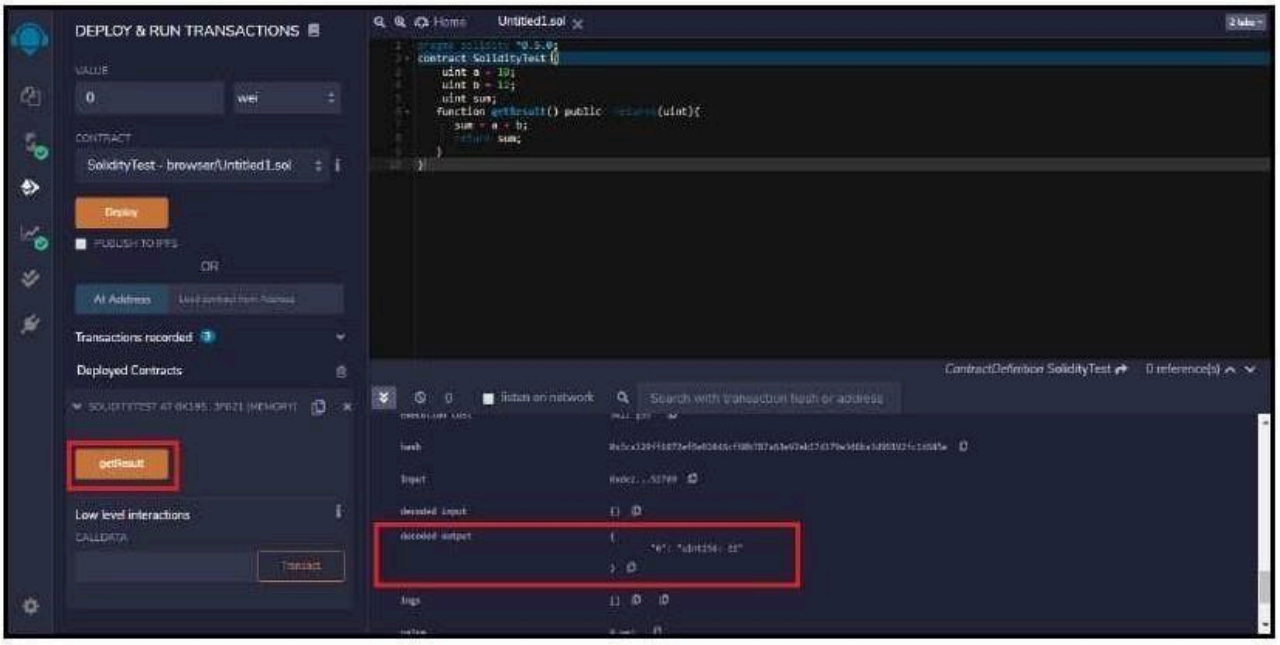
deployed contracts to run the program, and for output, check to click on the drop-down on the console.

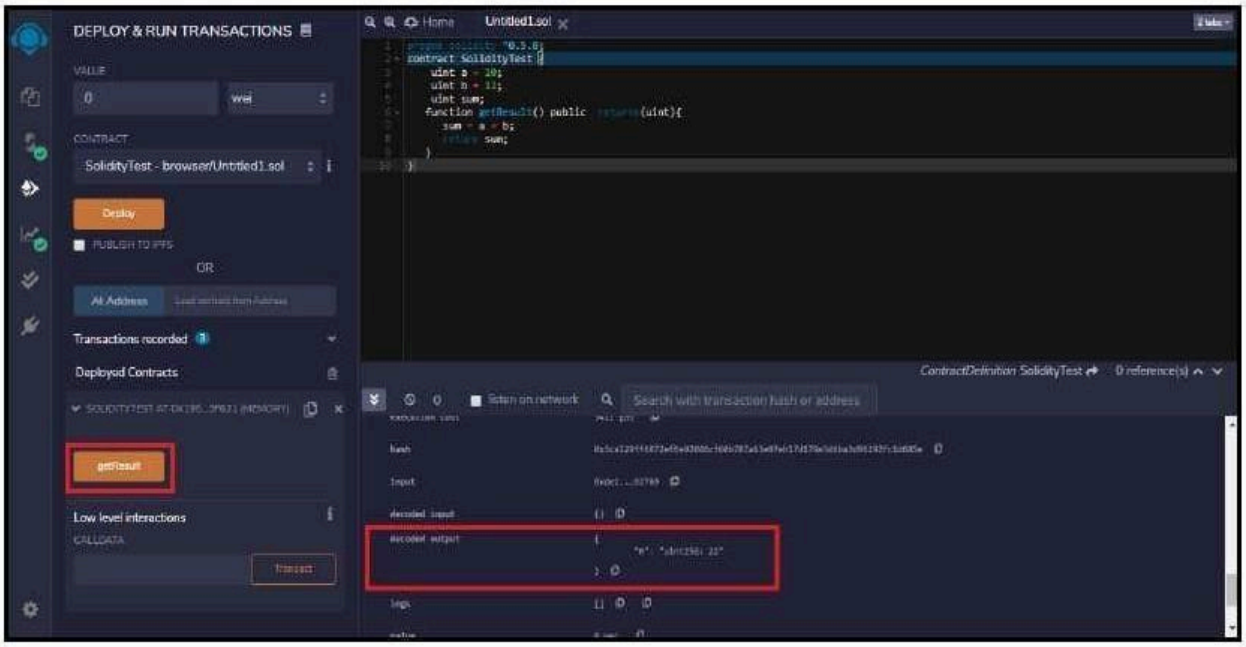
Step 5:

For debugging click on the

Debug button

corresponding to the method call in the console. Here you can check each function call and variable assignments.




DEPLOY & RUN TRANSACTIONS
VALUE
0
wei
CONTRACT
SolidityTest - browser/Untitled1.sol
Deploy
PUBLISH TO IPFS
OR
At Address
Transactions recorded
Deployed Contracts
getResult
Low level interactions
CALLDATA
Transact
Home
Untitled1.sol
pragma solidity ^0.8.0;
contract SolidityTest {
uint a = 10;
uint b = 11;
uint sum;
function getResult() public returns(uint){
sum = a + b;
return sum;
}
}
ContractDefinition SolidityTest
0 reference(s)
Listen on network
Search with transaction hash or address
hash
input
decoded input
decoded output
logs

Program -

Smart contract for the Bank account of the customer to do operations like Deposit, Withdraw and Show Balance

```
// SPDX-License-Identifier: GPL-3.0


pragma solidity >=0.7.0 <0.9.0;


contract SimpleBank {


mapping (address => uint) private balances;


address owner;


// Constructor is "payable" so it can receive ether,

constructor() public payable {


/* Set the owner to the creator of this contract */
```

```
owner = msg.sender;

}

 /// @notice Deposit ether into bank, requires method is "payable"

 /// @return The balance of the user after the deposit is made

function deposit(uint depoamount) public payable returns (uint) {

balances[msg.sender] += depoamount;

payable(msg.sender).transfer(depoamount);

 //emit LogDepositMade(msg.sender, msg.value);

return balances[msg.sender];

}
```

```
 /// @notice Withdraw ether from bank

 /// @return The balance remaining for the user

function withdraw(uint withdrawAmount) public returns (uint) {

 // Check enough balance available, otherwise just return balance

if (withdrawAmount <= balances[msg.sender]) {

balances[msg.sender] -= withdrawAmount;

payable(msg.sender).transfer(withdrawAmount);

}

return balances[msg.sender];

}
```

```
function balance() public view returns (uint) {

return balances[msg.sender];

}

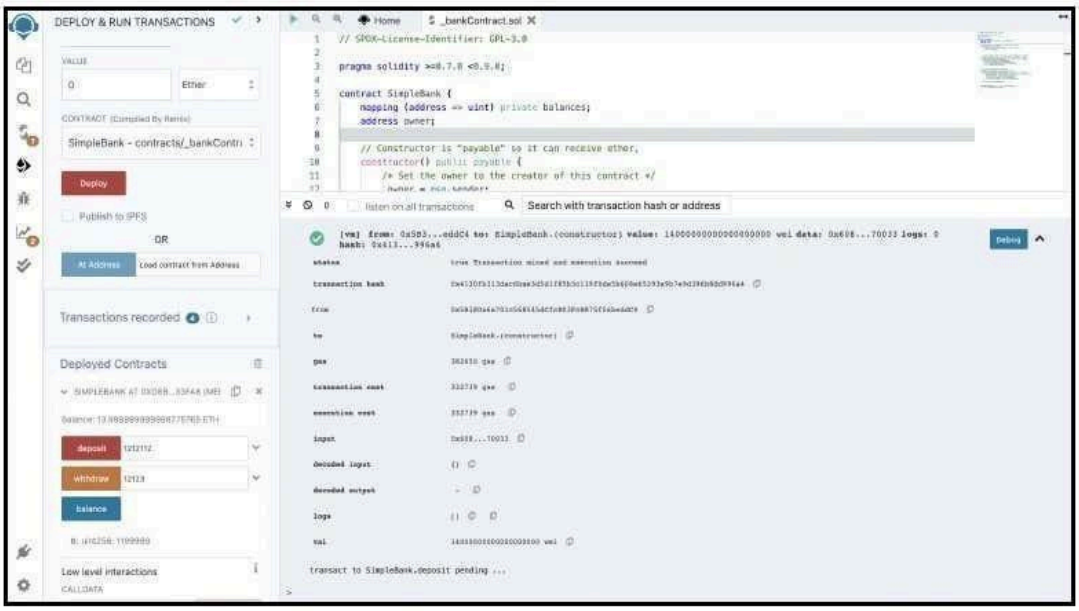
}
```

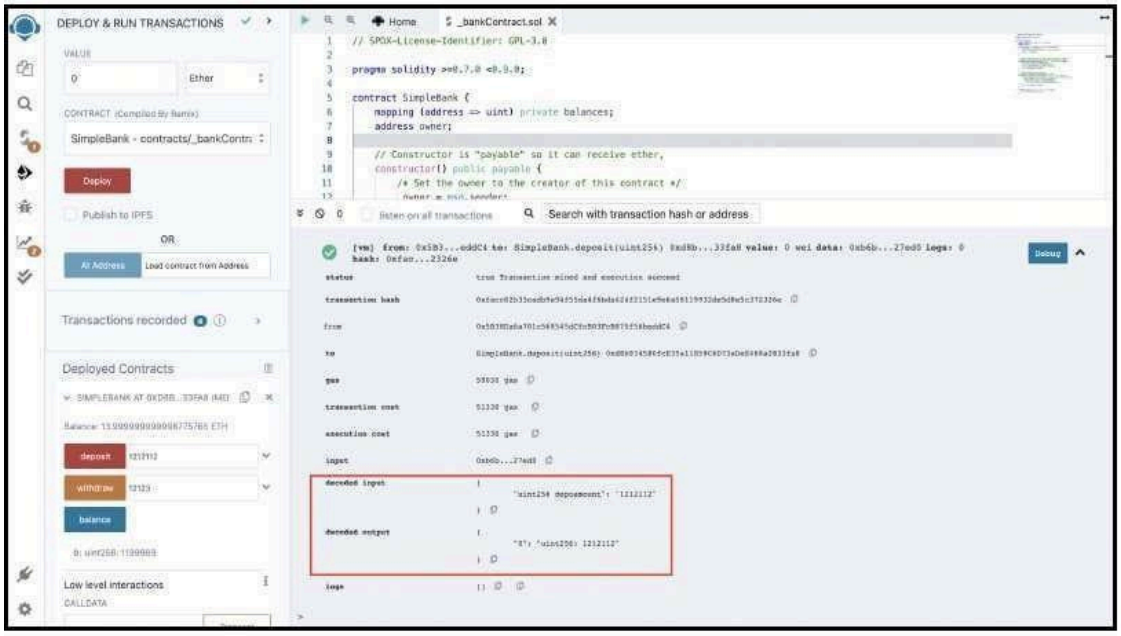
Output -

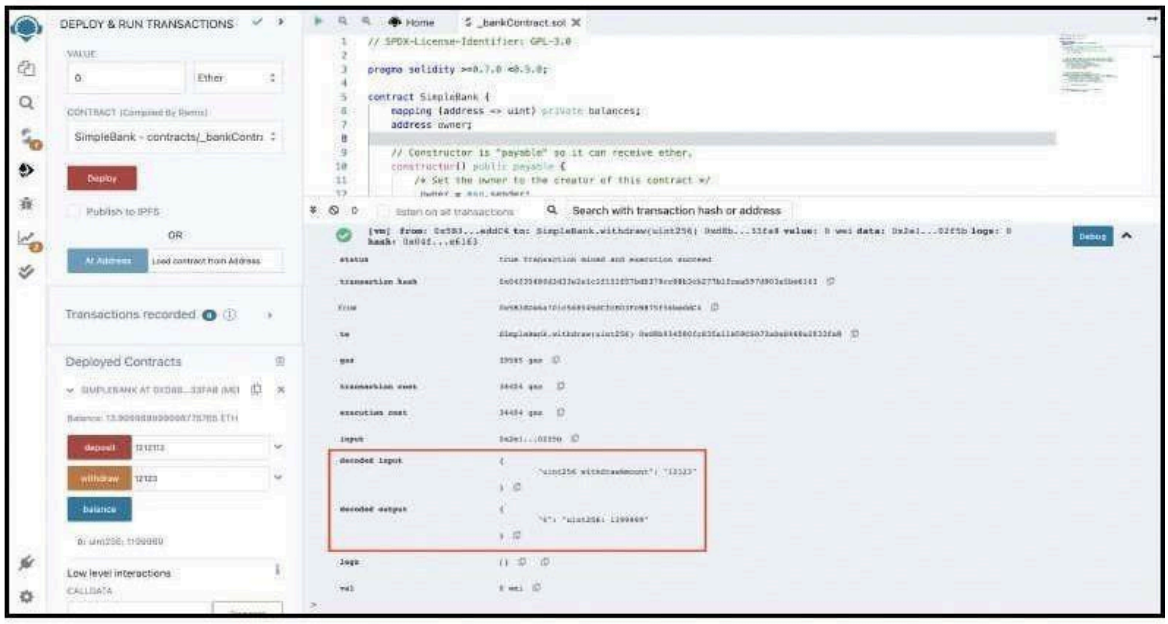
Deploy

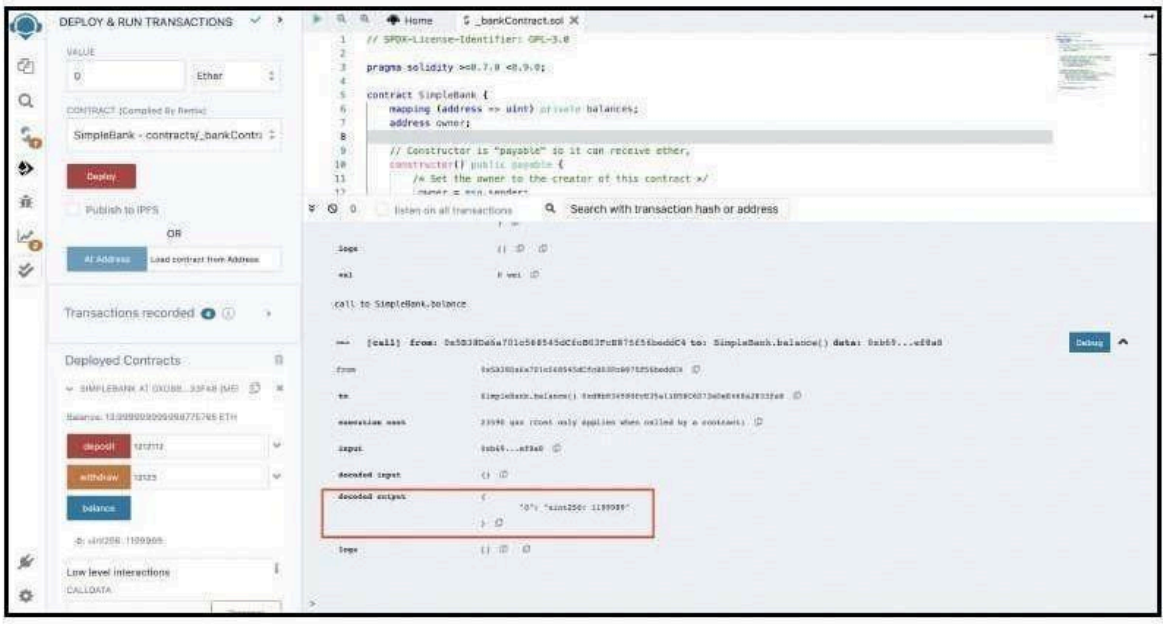
Deposit

Money


DEPLOY & RUN TRANSACTIONS
VALUE
0
Ether
CONTRACT (Compiled By Remix)
SimpleBank - contracts/_bankContr
Deploy
Publish to IPFS
OR
At Address
Load contract from Address
Transactions recorded
Deployed Contracts
deposit
withdraw
balance
Low level interactions
CALLDATA
Home
_bankContract.sol
// SPDX-License-Identifier: GPL-3.0
pragma solidity >=0.7.0 <0.9.0;
contract SimpleBank {
mapping (address => uint) private balances;
address owner;
// Constructor is "payable" so it can receive ether,
constructor() public payable {
/* Set the owner to the creator of this contract */
Search with transaction hash or address
Debug
transact to SimpleBank.deposit pending ...


DEPLOY & RUN TRANSACTIONS
VALUE
0
Ether
CONTRACT (Compiled By Remix)
SimpleBank - contracts/_bankContr
Deploy
Publish to IPFS
OR
At Address
Load contract from Address
Transactions recorded
Deployed Contracts
deposit
withdraw
balance
Low level interactions
CALLDATA
Home
_bankContract.sol
// SPDX-License-Identifier: GPL-3.0
pragma solidity >=0.7.0 <0.9.0;
contract SimpleBank {
mapping (address => uint) private balances;
address owner;
// Constructor is "payable" so it can receive ether,
constructor() public payable {
/* Set the owner to the creator of this contract */
Search with transaction hash or address
Debug
decoded input
decoded output
logs


DEPLOY & RUN TRANSACTIONS
VALUE
0
Ether
CONTRACT (Compiled By Remix)
SimpleBank - contracts/_bankContr
Deploy
Publish to IPFS
OR
At Address
Load contract from Address
Transactions recorded
Deployed Contracts
deposit
withdraw
balance
Low level interactions
CALLDATA
Home
_bankContract.sol
// SPDX-License-Identifier: GPL-3.0
pragma solidity >=0.7.0 <0.9.0;
contract SimpleBank {
mapping (address => uint) private balances;
address owner;
// Constructor is "payable" so it can receive ether,
constructor() public payable {
/* Set the owner to the creator of this contract */
Search with transaction hash or address
Debug
status
transaction hash
from
to
gas
transaction cost
execution cost
input
decoded input
decoded output
logs
val


DEPLOY & RUN TRANSACTIONS
VALUE
0
Ether
CONTRACT (Compiled By Remix)
SimpleBank - contracts/_bankContr
Deploy
Publish to IPFS
OR
At Address
Load contract from Address
Transactions recorded
Deployed Contracts
deposit
withdraw
balance
Low level interactions
CALLDATA
Home
_bankContract.sol
// SPDX-License-Identifier: GPL-3.0
pragma solidity >=0.7.0 <0.9.0;
contract SimpleBank {
mapping (address => uint) private balances;
address owner;
// Constructor is "payable" so it can receive ether,
constructor() public payable {
/* Set the owner to the creator of this contract */
Search with transaction hash or address
logs
val
call to SimpleBank.balance
[call] from: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4 to: SimpleBank.balance() data: 0xb69...ef0a8
Debug
from
to
execution cost
input
decoded input
decoded output
logs